

Tìm chuỗi con lặp lớn nhất

Lin Xide

2003

Nguồn gốc: Finding maximal repetition

IOI China candidate team papers / 2003 / Lin Xide / Lin Xide.doc

Abstract

So với nhiều chủ đề thuật toán khác, xử lý chuỗi ít được trình bày trong các tài liệu tiếng Trung cổ điển về thi tin học. Bài viết này mô tả bài toán tìm chuỗi con lặp lớn nhất, rồi lần lượt nhắc lại hai công cụ thường dùng trong xử lý chuỗi: cây hậu tố và KMP. Trên cơ sở đó, bài viết trình bày một thuật toán tuyến tính để tìm chuỗi con lặp có chu kỳ lớn nhất.

Từ khóa: cây hậu tố, mở rộng KMP, chuỗi con tối ưu.

1 Bài toán và nhận xét ban đầu

1.1 Định nghĩa

Với chuỗi S , nếu tồn tại số nguyên dương p sao cho

$$1 \leq x \leq |S| - p \implies S_x = S_{x+p},$$

thì gọi p là một **chu kỳ** của S . Khi đó

$$e = \frac{|S|}{p}$$

được gọi là **số mũ** của S , và mọi chuỗi con độ dài p của S đều có thể xem là một **chuỗi sinh** của S . Đặc biệt, nếu $e \geq 2$, ta gọi S là một chuỗi lặp kích thước p .

Bài toán: cho một chuỗi W gồm các chữ cái in hoa, độ dài

$$1 \leq n \leq 10^5,$$

hãy tìm trong mọi chuỗi con của W một chuỗi lặp có chu kỳ lớn nhất. Bài toán này được gọi là tìm **chuỗi con lặp lớn nhất**.

Ký hiệu $W(u, v)$ là chuỗi con của W bắt đầu tại vị trí u và kết thúc tại vị trí v .

1.2 Cách trực tiếp

Thuật toán trực tiếp là duyệt chu kỳ p từ lớn xuống nhỏ, rồi với mỗi p quét toàn bộ chuỗi để xem có hai khối liên tiếp độ dài p giống nhau hay không. Khi tìm thấy, có thể trả lời ngay vì ta đang xét chu kỳ theo thứ tự giảm dần.

Ý tưởng này dễ cài đặt nhưng có độ phức tạp $O(n^2)$. Để giảm xuống tuyến tính, ta cần hai công cụ: cây hậu tố để giới hạn dạng xuất hiện của nghiệm, và một biến thể KMP để tính nhanh các độ dài khớp tiền tố/hậu tố.

2 Cây hậu tố

2.1 Định nghĩa

Trước hết thêm ký tự kết thúc đặc biệt vào cuối chuỗi:

$$W \leftarrow W\$,$$

trong đó \$ không xuất hiện ở vị trí nào trước đó. Ký tự này bảo đảm không có hậu tố nào là tiền tố của hậu tố khác.

Cây hậu tố là một trie nén chứa tất cả các hậu tố của W . Mỗi cạnh $E(u, v)$ được gắn bởi một cặp chỉ số

$$E(u, v) = (\ell, r),$$

biểu diễn chuỗi con $W(\ell, r)$. Với bảng chữ cái $A, \dots, Z$, mỗi nút có tối đa 26 con tương ứng với ký tự đầu của nhãn cạnh.

Nếu nối các nhãn cạnh trên đường từ gốc tới nút x , ta được một chuỗi S_x . Ta gọi S_x là chuỗi tương ứng của x , và x là nút tương ứng của S_x . Mỗi hậu tố $W(i, n)$ tương ứng với một lá $Leaf_i$.

2.2 Dựng cây

Khi chèn hậu tố $W(i, n)$, đặt $Head_i$ là tiền tố chung dài nhất giữa $W(i, n)$ và một hậu tố $W(j, n)$ đã chèn trước đó, với $j < i$. Đặt $Tail_i$ sao cho

$$Head_i + Tail_i = W(i, n).$$

Nói cách khác, $Head_i$ là tiền tố dài nhất của $W(i, n)$ đã xuất hiện trong phạm vi $i - 1$. Sau khi tìm được nút tương ứng với $Head_i$, ta chỉ cần thêm một lá mới cho phần còn lại.

Cách tìm thô là từ gốc so sánh từng ký tự của $W(i, n)$. Nếu đi hết một cạnh thì tiếp tục xuống nút con; nếu dừng giữa cạnh thì tách cạnh và tạo nút mới. Cách này đúng nhưng có thể tốn $O(n^2)$.

2.3 Liên kết hậu tố

McCreight dùng **liên kết hậu tố** để tăng tốc. Giả sử

$$Head_{i-1} = az,$$

trong đó a là ký tự đầu. Vì z đã xuất hiện ít nhất hai lần trong phạm vi đã xét, $Head_i$ có tiền tố z . Liên kết hậu tố của nút ứng với az trở tới nút ứng với z . Nếu z rỗng, nó trở về gốc.

Khi cần tìm nút của $Head_i$, thay vì bắt đầu từ gốc, ta đi theo liên kết hậu tố của nút trước, rồi chỉ tìm tiếp phần còn thiếu. Khi tạo một nút mới, liên kết hậu tố của nó cũng được tạo ngay bằng cách đi từ liên kết hậu tố của cha và đi xuống theo nhãn cạnh tương ứng.

Điểm then chốt trong phân tích là đại lượng

$$i + |Head_i|$$

không giảm khi i tăng. Vì vậy toàn bộ số ký tự mà thủ tục tìm kiếm duyệt qua là $O(n)$. Cây hậu tố có thể được dựng trong $O(n)$ thời gian và $O(n)$ bộ nhớ.

3 KMP và mở rộng KMP

3.1 KMP cổ điển

Cho chuỗi chính S và mẫu T , với $n = |S|$, $m = |T|$. Thuật toán khớp mẫu trực tiếp có thể tốn $O(nm)$, vì khi gặp một ký tự khác nhau nó quay lại quá nhiều vị trí đã biết thông tin.

KMP cải thiện điểm này bằng hàm tiền tố. Đặt $\pi[j]$ là độ dài lớn nhất của một tiền tố của T đồng thời là hậu tố của $T(1, j)$, nhưng ngắn hơn toàn bộ $T(1, j)$. Khi đang so sánh tới T_j và S_i mà không khớp, ta dịch mẫu sao cho phần đã khớp dài nhất còn giữ được, tức chuyển về $\pi[j - 1] + 1$ thay vì bắt đầu lại từ đầu.

Hàm π được tính trong $O(m)$, và thuật toán khớp mẫu chạy trong $O(n + m)$.

3.2 Mở rộng KMP

Trong bài toán chính, ta không chỉ cần biết mẫu có xuất hiện hay không, mà cần nhiều giá trị độ dài khớp. Định nghĩa

$$B_i = \text{lcp}(T, S(i, n)), \quad 1 \leq i \leq n,$$

tức độ dài tiền tố chung dài nhất của T và hậu tố $S(i, n)$.

Để tính nhanh B_i , trước hết tính mảng

$$A_i = \text{lcp}(T, T(i, m)), \quad 2 \leq i \leq m.$$

Đây chính là dạng thường gọi là Z-function. Giữ một vị trí k sao cho $k + A_k$ xa nhất trong các vị trí đã biết. Khi tính A_i , đặt

$$Len = k + A_k - 1, \quad L = A_{i-k+1}.$$

Nếu $i \leq Len$ và $L < Len - i + 1$, ta có ngay

$$A_i = L.$$

Ngược lại, bắt đầu từ phần đã biết và tiếp tục so sánh từng ký tự để mở rộng A_i , rồi cập nhật $k = i$.

Vì biên phải $k + A_k$ chỉ tăng, tổng số lần so sánh ký tự là $O(m)$. Mảng B được tính tương tự bằng cách dùng mảng A làm thông tin tham chiếu; độ phức tạp là $O(n)$. Do đó mở rộng KMP tính toàn bộ các độ dài khớp cần thiết trong $O(n + m)$.

4 Thuật toán chính

4.1 Chuỗi con tối ưu

Với chuỗi con $S = W(u, v)$ và số nguyên dương L , nếu:

- S là chuỗi lặp có chu kỳ L ;
- S không thể mở rộng sang trái mà vẫn có chu kỳ L , tức $u = 1$ hoặc $W(u - 1, v)$ không còn thỏa điều kiện;
- S không thể mở rộng sang phải mà vẫn có chu kỳ L , tức $v = n$ hoặc $W(u, v + 1)$ không còn thỏa điều kiện,

thì gọi S là một **chuỗi con tối ưu** có chu kỳ L .

Nếu tìm được tất cả chuỗi con tối ưu và chu kỳ của chúng, bài toán ban đầu trở thành chọn chuỗi có chu kỳ lớn nhất. Ý tưởng chung là:

1. tìm một chuỗi sinh D độ dài L ;
2. mở rộng D sang trái và sang phải đến khi không thể mở rộng nữa;
3. nếu chuỗi sau mở rộng có độ dài ít nhất $2L$, ta thu được một chuỗi con tối ưu chu kỳ L .

4.2 Phân rã chuỗi

Phân rã W thành

$$W = U_1 + U_2 + \cdots + U_m$$

theo quy tắc sau. Giả sử đã có

$$P = U_1 + \cdots + U_{i-1}, \quad W = P + Q.$$

Nếu ký tự đầu Q_1 chưa từng xuất hiện trong phạm vi $|P|$, đặt

$$U_i = Q_1.$$

Ngược lại, đặt U_i là tiền tố dài nhất của Q đã xuất hiện trong phạm vi $|P|$.

Ví dụ với

$$W = \text{ABAABABAABAAB},$$

phân rã thu được

$$|A|B|A|AB|ABAAB|AAB|.$$

Phân rã này được tính bằng cây hậu tố: U_i hoặc là một ký tự đơn, hoặc là $\text{Head}_{|P|+1}$. Vai trò của phân rã là giới hạn mạnh vị trí của chuỗi con tối ưu.

4.3 Giới hạn vị trí nghiệm

Xét một chuỗi con tối ưu S , và giả sử vị trí kết thúc của nó nằm trong mảnh U_i . Có thể bỏ qua trường hợp S bắt đầu trong chính U_i , vì khi đó S đã xuất hiện trong phần P theo định nghĩa của U_i , và nó sẽ được xét ở một mảnh trước.

Gọi R là chuỗi sinh cuối cùng của S , tức đoạn độ dài L ở cuối S . Bằng các lập luận phản chứng dựa trên định nghĩa phân rã, ta thu được hai kết luận:

- R không thể bắt đầu quá xa về bên phải của U_{i-1} ;
- điểm bắt đầu của S cũng không thể nằm quá xa bên trái so với $U_{i-1} + U_i$.

Kết luận gọn là: nếu đặt $U = U_i$, và đặt V là hậu tố của

$$P = U_1 + \cdots + U_{i-1}$$

có độ dài

$$|U| + 2|U_{i-1}|,$$

thì mọi chuỗi con tối ưu kết thúc trong U đều có điểm bắt đầu nằm trong V , còn điểm kết thúc nằm trong U .

Đây là ý nghĩa chính của phân rã: nó biến việc tìm kiếm trên toàn bộ chuỗi thành các bài toán cục bộ trên $V + U$.

4.4 Tìm chuỗi sinh

Với cặp V, U cố định, xét chu kỳ L . Vì chuỗi con tối ưu S bắt đầu trong V , kết thúc trong U , và có độ dài ít nhất $2L$, ít nhất một trong hai điều kiện sau đúng:

- phần của S nằm trong V có độ dài ít nhất L ;
- phần của S nằm trong U có độ dài ít nhất L .

Hai trường hợp đối xứng nhau. Xét trường hợp thứ hai: khi đó

$$U(1, L)$$

là một chuỗi sinh của S . Chỉ cần mở rộng chuỗi sinh này sang trái và sang phải để kiểm tra xem nó tạo được chuỗi lặp dài ít nhất $2L$ hay không.

4.5 Hàm phụ trợ

Để mở rộng trong $O(1)$ trung bình, định nghĩa:

$$Lp_L = \text{lcp}(U, U(L+1, |U|)),$$

và

$$Ls_L = \text{lcs}(V, V + U(1, L)),$$

trong đó lcs là độ dài hậu tố chung dài nhất. Trực tiếp tính từng giá trị có thể chậm, nhưng toàn bộ các giá trị Lp_L và Ls_L có thể được tính bằng mở rộng KMP trên các chuỗi thích hợp. Vì vậy chi phí bình quân cho mỗi L là $O(1)$.

Thuật toán tổng quát:

1. Dùng cây hậu tố để phân rã W thành $U_1, \dots, U_m$.
2. Với mỗi $i \geq 2$, đặt $U = U_i$, V là hậu tố của phần trước đó có độ dài $|U| + 2|U_{i-1}|$.
3. Tính các mảng mở rộng Lp và Ls cho cặp V, U .
4. Với mọi $1 \leq L \leq |U|$, dùng $U(1, L)$ làm chuỗi sinh, mở rộng sang hai phía bằng Lp_L, Ls_L , rồi kiểm tra độ dài sau mở rộng có ít nhất $2L$ hay không.
5. Cập nhật đáp án bằng chu kỳ lớn nhất tìm được.

4.6 Độ phức tạp

Bước phân rã dùng cây hậu tố nên tốn $O(n)$ thời gian và bộ nhớ. Với mỗi cặp V, U , các mảng phụ trợ được tính trong $O(|V| + |U|)$. Tổng độ dài các mảnh U_i là $O(n)$, và do cách chọn V , tổng chi phí trên mọi mảnh cũng tuyến tính. Việc liệt kê các chu kỳ L trong mỗi U có tổng chi phí

$$\sum_i O(|U_i|) = O(n).$$

Vì vậy toàn bộ thuật toán chạy trong $O(n)$ thời gian và dùng $O(n)$ bộ nhớ.

5 Kết luận

Cây hậu tố và KMP đều là những công cụ tinh tế nhưng không xa lạ. Điều đáng chú ý trong bài toán này là cách kết hợp chúng: cây hậu tố cho ta một phân rã có khả năng giới hạn hình dạng của nghiệm, còn mở rộng KMP giúp kiểm tra mọi chu kỳ cục bộ với chi phí tuyến tính. Bản thân thuật toán tìm chuỗi con lặp lớn nhất có thể không phải kỹ thuật thường xuyên dùng nguyên dạng, nhưng cách phối hợp nhiều công cụ xử lý chuỗi để biến một tìm kiếm bậc hai thành tuyến tính là kinh nghiệm đáng học.

References

- [1] Fu Qingxiang, Wang Xiaodong. *Algorithms and Data Structures*, second edition.
- [2] Donald E. Knuth. *The Art of Computer Programming*.
- [3] *Handbook of Computer Science and Engineering*.
- [4] Roman Kolpakov. *Finding Maximal Repetition*.
- [5] *Discrete Applied Mathematics*, 1989.